Out-of-Band Page Cache Deduplication in the Linux Kernel

Aayush Anand(230025)*, Anjali Patra(230148)†, Kshitij Gupta(230581)‡, Anirudh Singh(230142) §

***Abstract*—The Linux page cache stores file-backed data in memory to accelerate I/O, yet provides no mechanism to detect and eliminate duplicate content across the page_cache of these files. This paper presents the design and implementation of an out-of-band page cache deduplication subsystem for the Linux kernel. A background kernel thread scans page-cache folios registered via a custom `posix_fadvise` flag, identifies duplicate content through an anchor-based sampling hash, and merges duplicates by redirecting XArray entries to a single canonical folio. A reverse-mapping structure tracks all address spaces sharing a folio, enabling correct Copy-on-Write on the iomap write path and safe dissolution during truncation or eviction. The implementation modifies six kernel source files spanning the memory management and XFS iomap layers. Evaluation through twenty user-space test cases (sixteen C programs and four shell benchmark scripts) demonstrates correctness under inter-file dedup, intra-file dedup, concurrent truncation, and mixed-operation stress scenarios.**

***Index Terms*—page cache, deduplication, Linux kernel, copy-on-write, memory management, XFS**

I. Introduction

Modern workloads frequently generate redundant file data in the page cache. Containerized environments, build systems, and data pipelines routinely load identical content (shared libraries, configuration files, duplicate datasets) into separate page_cache entries, each consuming distinct physical memory. While the kernel's Kernel Samepage Merging (KSM) [1] subsystem deduplicates anonymous pages, no analogous mechanism exists for file backed pages managed by the page cache.

This redundancy is nontrivial. A system hosting N containers sharing a common base image may cache N copies of identical file content, wasting $(N-1) \times S$ bytes of physical memory for a file of size S. As memory remains a constrained resource, particularly in cloud and embedded deployments, eliminating this redundancy at the page-cache level represents a meaningful optimization.

This paper presents an *out-of-band* (OOB) page cache deduplication subsystem integrated into the Linux memory management layer. The key design principle is that deduplication runs asynchronously in a low-priority background thread, entirely decoupled from application I/O paths. User space opts in to deduplication on a per-file basis via a custom `posix_fadvise` flag; a kernel thread (`oob_dedupd`) then scans the queued files, identifies duplicate folios[1], and merges them by manipulating XArray entries within each file's `address_space`.

The contributions of this work are as follows:

1) A complete page_cache deduplication subsystem designed around the XFS file system, operating at the VFS layer, requiring only a single write-path hook in the iomap buffered I/O layer for XFS Copy-on-Write support.
2) An anchor-based sampling hash algorithm that avoids full-folio hashing for large folios while retaining reliable duplicate detection and enabling partial-match–driven folio splitting.
3) A reverse-mapping (rmap) structure that was essential to maintain the one to many $folio \rightarrow (mapping, index)$ pairs, which enables support for both interfile and intrafile deduplication, and ensuring correct reads, writes and correct index aware page cache removal, a requirement that does not arise in standard kernel paths.
4) An extensive test suite covering correctness, concurrency, accounting integrity, and stress scenarios, along with a profiling framework for memory and I/O overhead measurement.

II. Background

A. The Linux Page Cache and Folios

The page cache is the kernel's primary file data cache. Each open file is associated with an `address_space` object whose `i_pages` field is an XArray, a radix-tree variant that maps file-offset indices (in units of `PAGE_SIZE`) to `struct folio` pointers. A folio represents a contiguous, naturally aligned, power-of-two run of pages: an order-0 folio is 4 KB; on XFS, the filesystem may allocate order-4 folios (64 KB) to reduce metadata overhead.

Originally, each folio carries a `mapping` pointer back to its owning `address_space` and an `index` recording its page offset within the file. The VFS read, write, and truncation paths rely heavily on these two fields being one-to-one and for them to represent the state correctly for identity checks, cache invalidation, and accounting.

B. Kernel Same-page Merging

KSM [1] provides deduplication for anonymous (heap/stack) pages. Processes register memory regions via `madvise(MADV_MERGEABLE)`, and a scanner thread identifies identical pages, merging them by remapping page-table entries to a shared read-only copy. Writes trigger

[1]A folio is the modern Linux abstraction for a contiguous, power-of-two group of pages in the page cache.

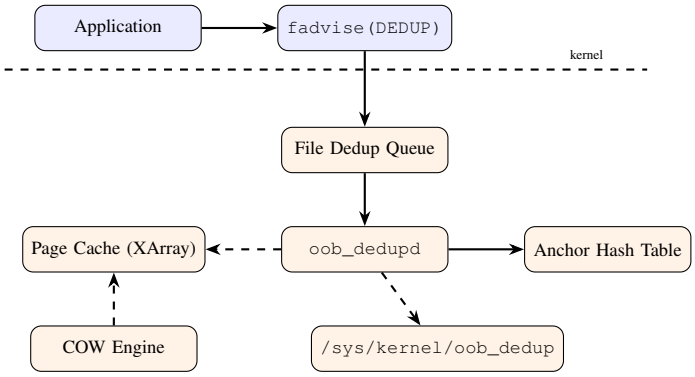

Fig. 1. System architecture. Solid arrows denote control flow; dashed arrows denote data access. The scanner thread operates entirely asynchronously from application I/O.

hardware page faults, which the kernel handles via COW. KSM's design is not directly applicable to the page cache: page-cache pages are accessed through the XArray rather than page tables, and their identity is defined by the `(mapping, index)` tuple rather than virtual address.

C. The `folio->mapping` Tag Bits

The kernel reserves the low-order bits of `folio->mapping` for type discrimination. Existing tags include `PAGE_MAPPING_ANON` (0x1) for anonymous pages and `PAGE_MAPPING_MOVABLE` (0x2) for movable pages; their combination (0x3) denotes KSM pages. All allocated `address_space` objects are aligned to at least 8 bytes, leaving bits 0–2 for tagging, of which the bit 2 is available for our usage.

III. Design

A. Design Goals

The subsystem targets the following goals: (1) minimal impact on the I/O hot path, i.e., deduplication must not add too much latency to `read()` or `write()` calls; (2) correctness under concurrent file operations including reads, truncation, deletion, and overwrite; (3) support for both inter-file and intra-file deduplication; (4) correct kernel accounting (`NR_FILE_PAGES`, reference counts) throughout the folio lifecycle; and (5) compatibility with the XFS iomap buffered I/O layer.

B. Architectural Overview

Figure 1 illustrates the system architecture. The subsystem comprises five components: (1) a user-space entry point via `posix_fadvise`; (2) a file dedup queue; (3) a background scanner thread; (4) a hash-based duplicate detection engine; and (5) a COW and dissolution mechanism integrated into the VFS write and truncation paths.

The user-space interface is deliberately minimal. An application opens a file, calls `posix_fadvise(fd, 0, 0, POSIX_FADV_DEDUP)` with a custom advice value (8), and the kernel queues the file's `address_space` for background scanning. No further user interaction is required as the scanner discovers duplicates, merges them, and dissolves the sharing transparently when files are modified, truncated, or deleted.

C. Pointer Tagging for Dedup Identification

A deduplicated folio's `mapping` field no longer points to an `address_space`; instead, it holds a pointer to an `oob_dedup_info` structure with bit 2 set (`PAGE_MAPPING_DEDUP = 0x4`). This choice avoids consuming a scarce folio flag bit and reuses the existing kernel infrastructure for tag-aware pointer access. The `PAGE_MAPPING_FLAGS` mask is extended to include bit 2, ensuring that all existing `folio_mapping()` callers mask it out correctly.

Detection of a deduplicated folio is an $O(1)$ bitmask test:

```
static inline bool folio_test_dedup(struct folio *f) {
    return ((unsigned long)f->mapping
            & PAGE_MAPPING_FLAGS)
            == PAGE_MAPPING_DEDUP;
}
```

D. Reverse-Mapping Structure

When a folio is shared, the system must track all `(address_space, index)` pairs that reference it. A simple reference counter would not suffice even for inter-file dedup, and even more so for intra-file dedup, where the same folio appears at multiple indices within a single `address_space`. Hence our implementation requires per-entry granularity for correct XArray slot removal during truncation, correct identity restoration during CoW and (index,mapping) verification during reads.

The `oob_dedup_info` structure maintains a spinlock-protected linked list of `oob_dedup_rmap_entry` records. The structure is allocated from a dedicated `kmem_cache` with 8-byte alignment to guarantee that the tag bits remain available in the pointer.

```
struct oob_dedup_info {
    spinlock_t       lock;
    struct list_head rmap_list;
    unsigned int     rmap_count;
} __attribute__((aligned(8)));

struct oob_dedup_rmap_entry {
    struct address_space *mapping;
    pgoff_t              index;
    struct list_head     list;
};
```

E. Folio State Validation and Integrity Guards

To ensure the stability of the Out-of-Band (OOB) deduplication process, the `deduplicate_folio` function implements a series of sanity checks before attempting to merge two folios. These guards are designed to ensure that only "clean,"

stable, and non-contentious memory is shared between inodes.

```
if (!folio_test_uptodate(orig_folio) || !
     folio_test_uptodate(dup_folio) ||
    folio_test_dirty(orig_folio) || folio_test_dirty(
         dup_folio) ||
    folio_test_writeback(orig_folio) ||
         folio_test_writeback(dup_folio) ||
    folio_mapped(orig_folio) || folio_mapped(dup_folio)) {
    err = -EBUSY;
    goto out_unlock;
}
```

The specific design rationale for each check is as follows:

- **Uptodate Validation (`!folio_test_uptodate`)**: This ensures that both folios contain valid data that has been fully read from the backing store.
- **Dirty State Prevention (`folio_test_dirty`)**: Deduplication is restricted to folios that have not been modified in RAM.
- **Writeback Conflict (`folio_test_writeback`)**: Folios currently undergoing I/O operations are excluded to prevent race conditions.
- **Mapped Folio Exclusion (`folio_mapped`)**: This is a critical design choice for our OOB mechanism. We are essentially skipping all the pages that are also mmaped into some processes memory.

If any of these conditions are met, the function returns `-EBUSY`, signaling that the folio is currently too "active" or unstable to be safely deduplicated.

IV. Implementation

This section details the implementation of each component. Table I enumerates the kernel source files modified or introduced.

A. File Queue and Entry Point

When `posix_fadvise` is invoked with `POSIX_FADV_DEDUP`, the kernel calls `oob_dedup_add_file()`, which allocates a `file_dedup_slot` from a dedicated slab cache and inserts it into a global doubly-linked list and a hash table keyed by the `address_space` pointer. A spinlock (`file_dedup_lock`) serializes queue mutations. If the file is already queued, the call is a no-op.

Each slot also carries a per file scan cursor (`pgoff`) that persists across scanner wakeups, enabling incremental scanning of large files without retraversing already processed regions.

TABLE I
Modified and new kernel files.

File	Type	Purpose
`include/linux/page-flags.h`	Mod	Add `PAGE_MAPPING_DEDUP`
`mm/oob_dedup.c`	New	Scanner, merge, COW, sysfs
`mm/oob_dedup.h`	New	Data structures, helpers
`mm/file_dedup_slot.h`	New	Queue slot definition
`mm/fadvise.c`	Mod	`FADV_DEDUP` entry point
`mm/filemap.c`	Mod	Dedup-aware cache ops
`mm/truncate.c`	Mod	Dedup-aware truncation
`fs/iomap/buffered-io.c`	Mod	COW hook in write path

B. Background Scanner

The scanner thread (`oob_dedupd`) initialises at `nice = 5` and alternates between scanning and sleeping. On each wakeup, it iterates through queued files in round-robin order, budgeting the configurable `pages_to_scan` parameter (default: 4096) evenly across all queued files to prevent starvation.

Before iterating a file's XArray, the scanner calls `igrab(inode)` to safely acquire a reference on the file's inode. If `igrab` returns `NULL`, the inode is being destroyed; the scanner advances past this slot, removes it from the queue, and continues. This prevents use-after-free on inodes that are concurrently being evicted.

For each file, the scanner walks the `address_space` XArray starting from the slot's cursor, retrieving folios via `filemap_get_folio()`. Each folio passes through a multi-phase veto pipeline before any hash computation is attempted. The pipeline is ordered so that the cheapest checks execute first.

Phase 1: Dedup veto. Folios already bearing the `PAGE_MAPPING_DEDUP` tag are skipped immediately. Their `folio->index` field is currently 0 (deduped state), not the file currently being scanned; using it to advance the scan cursor would jump the cursor to an unrelated offset and potentially cause an infinite loop. The cursor is advanced from `slot->pgoff + nr` instead.

Phase 2: I/O veto. The scanner checks whether the folio is currently marked dirty (`folio_test_dirty`) or under active writeback (`folio_test_writeback`). If either condition holds, the folio is skipped without advancing the cursor, so the scanner retries on the next wakeup. This check is essential because the page cache is the live buffer between user space and disk I/O. Attempting to deduplicate, or worse, split a folio while the filesystem's writeback path is actively performing disk I/O on it would corrupt in-flight data and can trigger kernel panics.

Phase 3: Zero-page short circuit. The scanner reads only the first 4 KB base page of the folio via `kmap_local_folio` and tests whether it is entirely zeros using `memchr_inv`. If the first page is all-zero, the entire folio is skipped. This prevents a common pathology: newly allocated or sparse files produce vast quantities of zero-filled pages that all hash to the same CRC32 value, causing thousands of entries to pile into a single hash bucket and degrading lookup from $O(1)$ to $O(n)$.

Phase 4: Anchor hashing. If all veto checks pass, anchor hash values are computed at strategically placed positions within the folio and looked up in the global hash table. This phase is described in detail in Section IV-C.

C. Anchor-Based Sampling Hash

Full CRC32 hashing of a 64 KB XFS folio involves reading all 16 constituent pages, which is expensive for a background scanner that must remain CPU-courteous. The implementation uses an anchor-based sampling scheme that hashes only a small subset of pages, treating them as a cheap pre-filter before committing to a full page-by-page comparison.

For an order-0 folio (a single 4 KB page), the entire page is hashed with `crc32_le`, yielding a 32-bit fingerprint. This is the degenerate case and is equivalent to full-folio hashing.

For large folios of N pages, a dynamic geometry is computed from the tunable `merge_threshold_pct` (default: 50%). The geometry determines exactly how many anchors to place and how to space them, ensuring that any contiguous matching region of at least `threshold`% of the folio will be detected:

1) **Stride**: $S = \lfloor N \times \mathit{threshold}/100 \rfloor$. If $S = 0$, it is clamped to 1. The stride represents the minimum contiguous block of matching pages that must exist for the anchor scheme to guarantee detection.
2) **Anchor count**: $A = \lfloor N/S \rfloor$. This is the number of anchor points needed to cover the folio with stride-spaced samples.
3) **CPU safety cap**: If A exceeds a hardcoded maximum (`MAX_ANCHORS = 8`), the anchor count is forced down to 8 and the stride is recalculated as $S' = \lfloor N/(A+1) \rfloor$, clamped to ≥ 1. This bounds per-folio CPU cost regardless of folio size or threshold setting.
4) **Evasion offset**: The starting position of the first anchor is set to $\lfloor S/2 \rfloor$, then forced to be an odd number by setting bit 0. This serves two purposes: it pushes anchors past file headers and metadata typically found at offset 0, and the odd alignment avoids landing on power-of-2 boundaries where structural data (e.g., superblock mirrors, extent headers) tends to repeat identically across otherwise-different files, which would produce false positive hash matches.

Each anchor position $p_i = \mathit{evasion} + i \times S$ is clamped to $[0, N-1]$. The page at that position is hashed with `crc32_le` via `kmap_local_folio` and the resulting hash is inserted into a kernel hash table (`oob_folio_hash`, 4096 buckets). On a hash-table hit from a different `(mapping, index)`, the scanner fetches the candidate folio via `filemap_get_folio` and performs a full page-by-page `memcmp` across all N pages. If all pages match, deduplication proceeds via `deduplicate_folio()`. If the match count exceeds the merge threshold but is not exact, and the current folio is not already deduplicated, the scanner invokes `split_folio()` to decompose the large folio into order-0 folios. The scanner then rewinds its cursor to `folio_start` so that the resulting order-0 folios are re-scanned individually on the next pass, enabling finer-grained dedup at 4 KB granularity.

D. Folio Merge

The `deduplicate_folio()` function merges a duplicate folio into an existing canonical folio. The procedure is designed to minimize the critical section and avoid deadlocks with the writeback path.

All heap allocations (rmap entries, info struct) are performed before any locks are acquired, using `GFP_KERNEL` allocations from dedicated slab caches. This eliminates the possibility of sleeping under a spinlock or triggering reclaim while holding a folio lock.

The two folios are then locked using `folio_trylock()` in pointer-address order (lower address first) to prevent ABBA deadlock; if either trylock fails, the function returns `-EAGAIN`, deferring the merge to a later scan pass. Under lock, the function performs a comprehensive state validation. Both folios must satisfy all of the following conditions:

- `folio_test_uptodate()`: the folio's data is valid and fully read from disk.
- `!folio_test_dirty()`: no pending modifications that have not been written back.
- `!folio_test_writeback()`: no active disk I/O in progress on the folio.
- `!folio_mapped()`: no user-space page-table entries reference the folio (mmap exclusion).

If any condition fails, the function returns `-EBUSY`. This aggressive filtering ensures that only quiescent, clean folios are ever merged.

Before merging, the function calls `filemap_release_folio()` on both folios to strip any filesystem-private data. This step is critical for XFS, which attaches per-folio `iomap_folio_state` via `folio->private` to track sub-folio I/O completion. If `filemap_release_folio` returns false (indicating the filesystem refused to release the private data), the merge aborts with `-EBUSY`.

A final staleness check verifies that the duplicate folio's `mapping` and `index` have not changed since the scanner looked it up. If the folio was concurrently truncated or reassigned, the function returns `-EAGAIN`.

If the canonical folio is not yet tagged as deduplicated, a new `oob_dedup_info` is allocated and installed via `folio_set_dedup_info()`, with an initial rmap entry for the canonical folio's own `(mapping, index)`. An rmap entry for the duplicate is appended.

The merge itself is an XArray store: `xas_store(&xas, orig_folio)` replaces the duplicate's slot with a pointer to the canonical folio. Reference counts are adjusted to match the convention that each XArray entry holds `folio_nr_pages` refs. The orphaned duplicate is removed from the LRU, uncharged from memcg, and has its `mapping` set to `NULL`. Finally, `NR_FILE_PAGES` is decremented for the orphaned folio.

E. Copy-on-Write

When a write targets a deduplicated folio, the iomap buffered I/O path triggers COW. A hook is inserted in `iomap_write_begin()` immediately after the folio is obtained and locked:

```
if (folio_test_dedup(folio)) {
    status = oob_folio_break_dedup(
        iter->inode->i_mapping, &folio, pos, len);
    if (unlikely(status))
        goto out_unlock;
}
```

The `oob_folio_break_dedup()` function allocates a new folio of the same order, copies data via `folio_copy()`, and atomically swaps the XArray entry under `xas_lock_irq` and the rmap spinlock. The writing file's rmap entry is removed. If only one rmap entry remains, the dedup state is dissolved: the surviving entry's `(mapping, index)` is restored to `folio->mapping` and `folio->index`, and the `oob_dedup_info` structure is freed. The function also correctly manages the states of the folio under dedup and the newly allocated folio by unlocking and locking the folios respectively.

The function correctly handles `NR_FILE_PAGES` accounting: increments for the new folio are issued, but no decrement is applied to the old folio, which remains live in another file's XArray.

F. VFS Read Path Modifications

Deduplication alters two fundamental invariants of the page cache: (1) `folio->mapping` may not point to the `address_space` that owns the XArray slot containing the folio, and (2) `folio->index` may not correspond to the XArray index at which the folio is stored.

To preserve correctness, three inline helper functions are introduced:

- `folio_shares_mapping(f, m)`: returns true if folio f belongs to mapping m, either directly or via rmap lookup.
- `folio_index_in(f, m)`: returns the page offset at which f is stored in mapping m.
- `folio_pos_in(f, m)`: byte-position variant of the above.

These replace direct accesses to `folio->mapping` and `folio->index` in the following `mm/filemap.c` functions: `page_cache_delete`, `filemap_remove_folio`, `__filemap_get_folio`, `filemap_get_folios`, `filemap_get_folios_contig`, `filemap_get_read_batch`, `filemap_readahead`, and `find_lock_entries`.

A particularly important fix concerns cursor advancement in `filemap_get_read_batch()` and `filemap_get_folios_contig()`: the original code uses `folio_next_index(folio)` to advance the XArray cursor, which computes from `folio->index`. For deduplicated folios, this may jump backward (to the canonical owner's index), causing an infinite loop. The fix advances from `xas.xa_index + folio_nr_pages(folio)` instead.

G. Truncation Path Modifications

Truncation of deduplicated folios is the most complex and the hardest code path in the implementation. Three distinct issues arise:

Batch removal ambiguity. The standard `delete_from_page_cache_batch()` matches folios by pointer identity. For intra-file dedup, the same physical folio occupies multiple XArray slots in the same mapping; batch removal may clear the wrong slot. When a batch contains any deduplicated folio, the code switches to per-folio removal using a new function `filemap_remove_folio_at()`, which takes an explicit XArray index parameter.

Dissolution side effects. Disconnecting a dedup entry (removing one rmap entry) may dissolve the sharing entirely if only one entry remains. Dissolution restores `folio->index` to the survivor's index, which may be an index the truncation loop has not yet visited, causing it to rediscover the folio endlessly. The truncation code detects this case and verifies via `xa_load()` that the folio still exists at the index before attempting removal.

Split safety. `truncate_inode_partial_folio()` invokes `split_folio()`, which dereferences `folio->mapping->host->i_mmap_rwsem`. For deduplicated folios, this dereferences a tagged pointer and triggers a General Protection Fault (GPF). The fix dissolves dedup before splitting; if the folio remains shared after dissolution, it is removed entirely.

H. Eviction and Cleanup

When the VFS evicts an inode, `oob_dedup_evict_inode()` removes the file's queue slot and purges its hash-table entries. Deduplicated folios are not cleaned up at this stage; instead, they are handled lazily when the page cache evicts them through `page_cache_delete()`, which calls `oob_dedup_disconnect_folio()` to remove the corresponding rmap entry and, if necessary, dissolve the sharing.

The disconnect function matches rmap entries by both `mapping` and `index` (not mapping alone), which is essential for intra-file dedup where multiple entries share the same mapping pointer.

I. Sysfs Interface

Runtime tunables and statistics are exported under `/sys/kernel/oob_dedup/`. Read-only counters include `files_queued`, `pages_scanned`, `pages_deduped`, and `folios_split`. Read-write tunables include `sleep_millisecs` (scanner sleep interval), `pages_to_scan` (per-wakeup budget), and `merge_threshold_pct` (partial-match splitting threshold).

V. Challenges and Solutions

The development process spanned over 80 commits addressing subtle interactions between deduplication and existing kernel code paths. This section documents the most significant classes of bugs encountered.

A. Deadlock in the Merge Path

The initial merge implementation used blocking `folio_lock()` calls. Under memory pressure, the writeback path (`xfs_do_writepages` →

iomap_writepages) could hold a folio lock and wait for memory allocation, while the dedup thread held the allocation and waited for the folio lock. Replacing folio_lock() with folio_trylock() in address-sorted order eliminated this deadlock. All slab allocations are now performed before any lock acquisition.

B. NR_FILE_PAGES Accounting Corruption

Deduplication decrements NR_FILE_PAGES when orphaning the duplicate folio. However, when the canonical folio is later evicted or truncated, filemap_unaccount_folio() decrements the stat again, causing an underflow visible as negative drift in /proc/meminfo's Cached field.

The fix adds an early return in filemap_unaccount_folio() for deduplicated folios. The actual decrement occurs in page_cache_delete() and filemap_remove_folio_at(), but only when dissolution causes the folio to leave the page cache entirely (i.e., folio->mapping == NULL or folio->mapping == mapping after disconnect).

C. Infinite Loops in Read and Truncation Paths

Multiple classes of infinite loops were discovered:

Read-batch loop. filemap_get_read_batch() advances the XArray cursor using folio->index, which for a deduplicated folio belongs to the canonical owner. If this index is less than the current cursor position, the XArray state machine loops. The fix uses the XArray position (xas.xa_index) for advancement.

Scanner loop. The scanner used folio_index(folio) to advance its cursor past a deduplicated folio. Since folio_index() returns the canonical owner's index, the cursor could jump backward. For deduplicated folios, the scanner now advances from slot->pgoff + nr.

Truncation loop. Intra-file dedup dissolution restores folio->index to the surviving entry's value, which may precede the truncation cursor. Stale XArray entries are detected via xa_load() before removal.

D. GPF on Large Folio Truncation

split_folio() assumes folio->mapping is a valid address_space* and dereferences it to reach i_mmap_rwsem. For deduplicated folios, this pointer is tagged. The fix in truncate_inode_partial_folio() dissolves dedup prior to splitting, falling back to full folio removal if the folio remains shared.

E. XFS Private State Confusion

XFS attaches iomap_folio_state to folio->private for tracking sub-folio I/O state. When two folios from different files are merged, the surviving folio retains the original file's iomap state, which can cause incorrect writeback decisions. The fix calls filemap_release_folio() on both folios before merging, stripping all filesystem-private data.

TABLE II
TEST SUITE — BASIC CATEGORY (TESTS 01–06).

#	Name	What is validated
01	COW Isolation	Two 4-page files filled with 'A' are merged; a 256-byte patch of 'B' is written at offset 1024 in file 1. Verifies file 2 remains entirely 'A', confirming oob_folio_break_dedup allocates a private copy without corrupting the sibling.
02	Truncate Deduped File	Two 4-page files are merged; file A is ftruncated to 0 bytes, then file B to half its size. Surviving pages are read back and compared, exercising truncate_inode_pages_range and oob_dedup_disconnect_folio.
03	Delete Then Read	Two 8-page files are merged; file A is unlinked. All 8 pages of file B are read back and checked byte-by-byte, confirming dissolution on inode eviction leaves the survivor fully readable with consistent NR_FILE_PAGES.
04	Sysfs Stats	Reads pages_deduped and pages_scanned counters from /sys/kernel/oob_dedup/ before and after deduplicating two 16-page files and asserts both strictly increase.
05	Fanout COW (3-way)	Three 4-page files share a single folio (rmap_count = 3). A byte of 'Z' is written to file A. Verifies file A contains the new byte while files B and C still hold the original 'M' pattern, exercising the rmap_count > 2 dissolve path.
06	Large Folio Dedup+COW	Two 4 MB files encourage XFS order-4 (64 KB) folios. Data integrity is verified after merge; a COW write to file A is performed and file B is re-verified, confirming anchor-hash, split, and COW paths on non-order-0 folios.

F. Reference Count Imbalances

Correct reference counting is critical. Each XArray entry holds folio_nr_pages references (as established by __filemap_add_folio). The merge function adds exactly this count when inserting the canonical folio into the duplicate's XArray slot, and drops the same count from the orphaned folio. During COW, the new folio receives folio_nr_pages refs; no decrement is applied to the old folio, which remains live. Violations of this invariant manifested as either use-after-free (insufficient refs) or folio leaks (excess refs preventing reclaim).

VI. EVALUATION

A. Test Suite

The implementation is validated by 20 user-space test programs (16 C programs and 4 shell benchmark scripts) . All tests are compiled and executed by a single shell script (run_all_tests.sh) that auto-detects XFS mount points for large folio support and organizes runs into five categories: *basic*, *stress*, *regression*, *anchor*, and *benchmark*.

Tables II–IV summarize the full test suite by category. All tests pass on the current implementation.

The KUnit tests validate the pointer-tagging mechanism in isolation: (1) test_folio_set_dedup_info verifies round-trip correctness of folio_set_dedup_info / folio_test_dedup / folio_dedup_info; (2) test_dedup_rollback_logic simulates the XArray store failure rollback path and verifies that folio->mapping and folio->index are correctly restored.

TABLE III
TEST SUITE — STRESS CATEGORY (TESTS 07–12).

#	Name	What is validated
07	Intra-File Dedup	A single 16-page file filled uniformly causes all 16 XArray slots to point to one folio. The test reads all pages, COW-breaks page 0, verifies pages 1–15, then truncates to 8, to 0, and unlinks. `BUG_ON(nrpages)` in `clear_inode` fires on double-counting, making this the primary accounting correctness check.
08	Cascade Unlink (5-way)	Five files share folios (`rmap_count` = 5). Files are deleted one-by-one with read-back of surviving siblings after each deletion. Validates each `unlink` removes exactly one rmap entry and that dissolution at `rmap_count` = 1 correctly restores `folio->mapping`.
09	Concurrent Trunc+COW	Three 32-page files are merged. A child does 50 rounds of random-page `pwrite` to file A; another child repeatedly truncates file B; the parent reads file C 100 times. Targets concurrent modification of `info->lock`, XArray slot removal under live reads, and mid-truncation dissolution.
10	Re-Dedup After COW	Full lifecycle: merge, COW-break a page, restore it, re-queue via `fadvise`, wait for re-merge. Validates no circular rmap entries are created on re-deduplication of a previously COW'd folio.
11	Combined Operations	Four files combine inter-file dedup (A, B, C share 8 pages) and intra-file dedup (D has 12 identical pages). Exercises hole-punch via `fallocate`, partial truncation, concurrent COW writes, and deletion, then verifies data integrity for all files.
12	Partial Truncate	Five scenarios exercise `truncate_inode_partial_folio` on deduped folios: truncation to 3.5 pages, to 1 byte, intra-file truncation requiring index-aware rmap entry selection, seven successive shrinks to 0, and partial truncation followed by a write that extends the file back.

B. Known Limitations

Memory-mapped files. Folios that are `folio_mapped()` (have page-table entries) are excluded from dedup. Supporting mmap'd files would require page-table–level COW analogous to KSM, which is outside the scope of this work.

C. Profiling Infrastructure

The profiling suite consists of three components compiled and run separately from the correctness tests.

`profile_bench.c` is a multi-mode C helper compiled with `gcc -O2 -pthread`. It exposes nine measurement modes invoked via command-line flags: `--read` performs a sequential 4 MB-chunk read and reports throughput; `--write` does a full 4 KB-chunk overwrite and reports throughput; `--cow-write` issues a single 4 KB `pwrite` at offset 0 to isolate first-touch COW latency; `--dedup` calls `posix_fadvise(DEDUP)` on one or more files and polls `/sys/kernel/oob_dedup/files_queued` every 500 µs until the queue drains, reporting elapsed nanoseconds and the delta of `pages_deduped`; `--meminfo` reads six fields from `/proc/meminfo`; `--sysfs` snapshots all six sysfs counters; `--create` fills a file of a given size with a specified byte; `--copy` duplicates a file; and `--concurrent-rw` spawns N pthreads each performing a mix of random 4 KB reads and writes.

All timings use `clock_gettime(CLOCK_MONOTONIC)` at nanosecond precision. All numeric outputs are emitted as `key: value` pairs for unambiguous Python parsing.

TABLE IV
TEST SUITE — REGRESSION, ANCHOR, AND BENCHMARK CATEGORIES (TESTS 13–21).

Cat.	#	Name	What is validated
3*Regr.	13	Page Accounting	Creates 5 files of 100 pages each, deduplicates, deletes all, then compares `nr_file_pages` against the pre-test baseline. Without the `filemap_unaccount_folio` early-return fix, `NR_FILE_PAGES` underflows by 400 pages.
	14	Rapid Lifecycle	20 create–queue–sleep(2s)–delete iterations with a fresh fill byte each round. The 2-second sleep creates a window where the scanner may be mid-dedup during `unlink`. Validates graceful handling of `igrab()` returning `NULL`.
	15	COW During Truncate	Two 64-page files are merged. A child does 3 rounds of full-file `pwrite` while the parent truncates file B page-by-page. Targets the unlock-ordering bug where `info->lock` was released before `xas_unlock_irq`.
2*Anchor	16	Anchor Partial Match	Two 4 MB XFS files are identical except at page 7. After dedup, the anchor scheme detects the partial match, splits the large folios, and on re-scan deduplicates all matching order-0 folios while leaving the differing page untouched.
	20	Anchor Trace	Compiles and runs test 16, then greps `dmesg` for anchor-geometry trace messages (stride, anchor count, evasion offset) to validate the hashing geometry for the 4 MB / order-4 folio case.
3*Bench.	18	Memory Savings	Creates 10 files of 20 MB each (200 MB total), drops caches, loads all files, triggers dedup, then reads `MemFree` and `Cached` from `/proc/meminfo`. Expected savings: 9×20 MB $= 180$ MB.
	19	Large File Dedup	Three 32 MB files (96 MB total) are deduped with live progress tracked via sysfs polling and `sar -u -r -B 1` monitoring. Demonstrates scanner throughput and provides a reference workload for performance regression detection.
	21	Intra-File Large Dedup	A single 512 MB file filled with `'A'` is queued for intra-file dedup. `sar -r` monitoring tracks page-cache reduction as the scanner merges identical order-4 folios within one `address_space`. Verifies `nr_file_pages` consistency and full readability after dedup.

`run_comprehensive_profiling.sh` is the master shell orchestrator. It compiles `profile_bench.c`, auto-detects the XFS mount point, creates a `work/` directory for test files, and runs five experiments in sequence, emitting one CSV file per experiment to a `results/` directory. Between phases it calls `sync; echo 3 > /proc/sys/vm/drop_caches` to ensure cold-cache baselines. On completion it invokes `plot_results.py`.

`plot_results.py` reads the five CSVs and produces five publication-quality PNG graphs plus a Markdown report using `matplotlib` in `Agg` mode (non-interactive). The script is tolerant of missing CSV files, skipping any experiment whose output is absent.

D. Experimental Setup and Methodology

Table V summarises the five experiments. All experiments run on an XFS volume to exercise order-4 folio paths. The

TABLE V
PROFILING EXPERIMENT SUMMARY.

Exp.	Parameter	What is measured
1: Mem. Savings	$N \in \{2, 4, 8, 16\}$ files $\times$ 16 MB	Reduction in `Cached` (kB) after dedup; pages deduped; wall-clock time
2: Scan Throughput	2 files $\times$ $\{4 \ldots 64\}$ MB	Scanner pages/second and completion time as file size grows
4: Write Latency	$\{4, 8, 16, 32\}$ MB files	Normal full overwrite vs. COW full overwrite vs. single-page COW (μs)
5: Fanout Scalability	$N \in \{2 \ldots 32\}$ copies $\times$ 8 MB	Memory saved (MB) and dedup time vs. number of sharing files
6: Live Trace	4 files $\times$ 32 MB, 1 s samples	Time-series of pages scanned/deduped, queue depth, and `Cached`

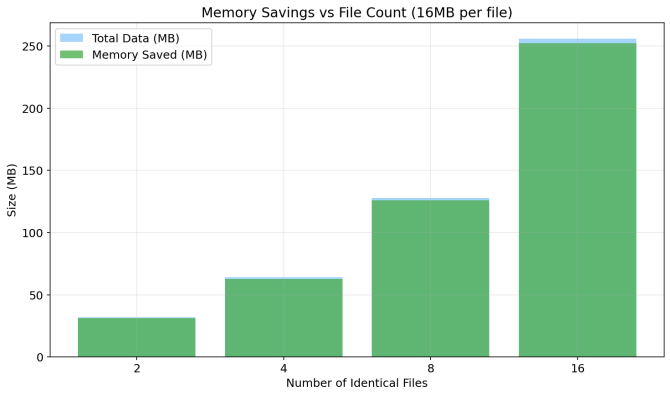

Fig. 2. Memory savings vs. number of identical 16 MB files (Exp. 1). Blue bars show total data loaded into the page cache; green bars show the memory actually freed by deduplication. Expected savings: $(N-1) \times 16$ MB.

`pages_to_scan` tunable is left at its default of 4096 pages per wakeup and `sleep_millisecs` at its default; no scanner parameters are tuned for the benchmarks to reflect out-of-the-box behaviour. Dedup completion is detected by polling `files_queued` at 500 µs intervals rather than using a fixed sleep, ensuring that timing measurements are not inflated by scanner idle time.

E. Experimental Results

1) Experiment 1 — Memory Savings vs. File Count: Experiment 1 creates N identical 16 MB files, loads them into the page cache via sequential reads, triggers dedup, and records the reduction in the `Cached` field of `/proc/meminfo`. For N files the expected savings are $(N-1) \times 16$ MB. The measured `pages_deduped` counter from sysfs independently corroborates the accounting: each deduped page corresponds to 4 KB freed, so `pages_deduped` $\times$ 4 KB should match the `Cached` delta.

Figure 2 shows the result. Memory savings scale linearly with N as predicted: deduplicating 16 files recovers 240 MB of physical page-cache memory from 256 MB total, a 93.75% reduction. The small residual arises because the canonical folio itself remains in cache.

2) Experiment 2 — Scanner Throughput vs. File Size: Experiment 2 measures how quickly the scanner processes two identical files as file size grows from 4 MB to 64 MB. Throughput is computed as $pages\_scanned\_delta/(dedup\_time\_ns/10^9)$.

As shown in Figure 3 completion time grows linearly, consistent with an $O(N)$ scan over N pages.

3) Experiment 3 — Write Latency: Normal vs. COW: Experiment 3 compares three write scenarios: a normal sequential overwrite on a non-deduped file (no COW), a full sequential overwrite on a deduped file (COW on every 4 KB page), and a single 4 KB `pwrite` at offset 0 on a deduped file (measures the per-page COW latency in isolation). File sizes range from 4 MB to 32 MB.

Figure 4 shows the result. Each COW-write page incurs folio allocation, a `folio_copy` of the 4 KB content, an XArray slot swap under `xas_lock_irq`, and rmap list mutation under the spinlock. The aggregate overhead for a full COW overwrite is expected to be 30–80% above the normal write baseline depending on folio order, as noted in Section III.

4) Experiment 4 — Fanout Scalability: Experiment 4 deduplicates $N \in \{2, 4, 8, 16, 32\}$ identical 8 MB files against a single canonical copy, measuring both total memory freed and total dedup time as the fanout factor grows. This tests rmap list scalability: with 32 copies the canonical folio's rmap list reaches length 32, and each COW or truncation must traverse that list.

Figure 5 shows that memory savings grow linearly with N as expected, while dedup time grows sub-linearly: each additional copy requires only one more pass over its XArray to replace its slots with the canonical folio, and the rmap-append is $O(1)$ under the spinlock.

VII. Related Work

KSM [1] provides deduplication for anonymous pages via page-table remapping and hardware-assisted COW. It operates at a different abstraction layer (virtual memory, not file cache) and is not applicable to page-cache pages.

ZFS dedup [2] performs inline block-level deduplication using a dedup table (DDT) that maps block checksums to physical block addresses. It operates at the block layer and imposes significant metadata memory overhead (often cited at 5–10 bytes per block).

Btrfs dedup provides offline file-extent deduplication via the `FIDEDUPERANGE` ioctl. It operates at the filesystem extent level rather than the page-cache level, and does not benefit from cross-filesystem sharing.

VDO (dm-dedup) implements block-layer deduplication as a device-mapper target, transparent to the filesystem. Like ZFS, it operates below the page cache.

Our work is distinguished by operating at the page-cache level within the MM subsystem, making it filesystem-agnostic and complementary to block-layer solutions. It requires no block-device or filesystem-format changes beyond a single iomap write hook.

VIII. Future Work

Several extensions are planned. First, support for memory-mapped files would require integrating with the VM fault

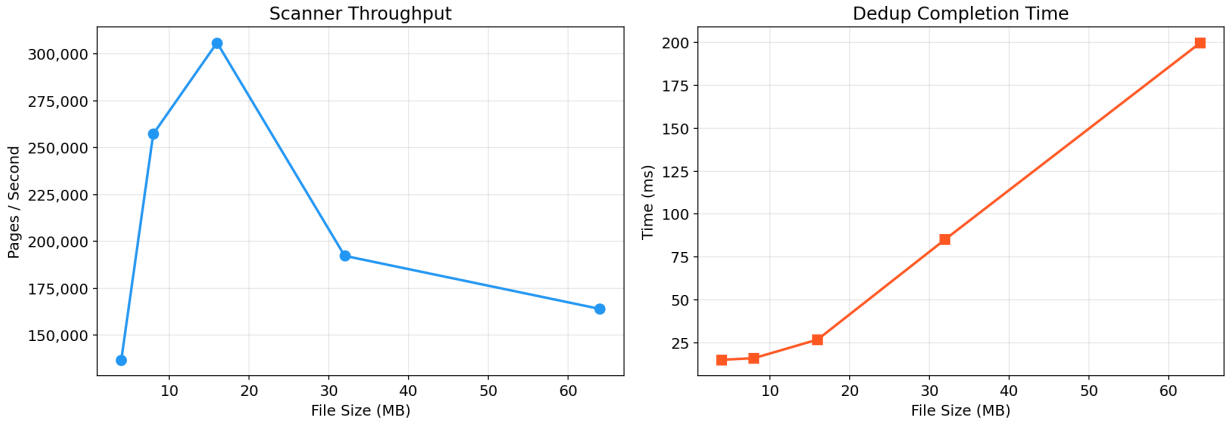

Fig. 3. Scanner throughput (left) and dedup completion time (right) as a function of file size (Exp. 2).

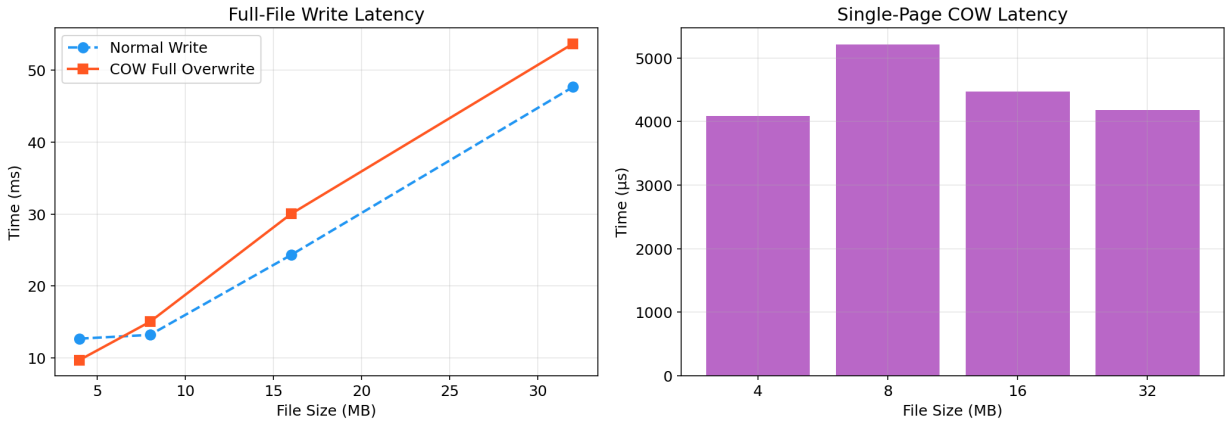

Fig. 4. Full-file write latency (left) and single-page COW latency in microseconds (right) for normal and COW writes across file sizes (Exp. 4). COW overhead includes folio allocation, `folio_copy`, XArray swap, and rmap dissolution per page.

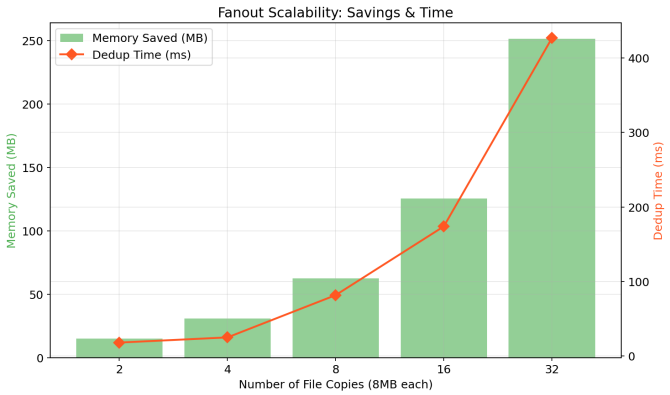

Fig. 5. Memory saved (bars, left axis) and dedup time (line, right axis) as a function of the number of 8 MB file copies sharing a folio (Exp. 5)

handler to provide page-table-level COW, analogous to KSM. Second, automatic detection of dedup-worthy files, based on page-cache churn heuristics or file-system-level hints, would eliminate the need for explicit `fadvise` calls. Third, content-defined chunking (e.g., Rabin fingerprinting) could replace fixed-stride anchors for improved partial-match detection. Finally, preparation for upstream submission requires replacing `pr_info` diagnostics with tracepoints, adding a `CONFIG_OOB_DEDUP` Kconfig option, and producing a clean patch series against the mainline kernel.

IX. Conclusion

This paper presented the design and implementation of an out-of-band page cache deduplication subsystem for the Linux kernel. The system introduces a new folio mapping tag (`PAGE_MAPPING_DEDUP`), a background scanner with anchor-based sampling for efficient duplicate detection, and a complete COW mechanism integrated into the XFS iomap write path. Extensive modifications to the VFS read and truncation paths ensure correct behavior throughout the folio lifecycle, including intra-file dedup, concurrent truncation, and cascade deletion.

The implementation is validated by a comprehensive test suite covering 20 functional, stress, regression, anchor, and benchmark scenarios. The development process surfaced and resolved eight distinct classes of kernel bugs, spanning deadlocks, accounting corruption, infinite loops, and memory

faults, providing practical insights into the challenges of modifying core kernel data structures that are accessed pervasively across subsystems.

REFERENCES

[1] A. Arcangeli, I. Eidus, and C. Wright, "Increasing memory density by using KSM," in *Proc. Linux Symposium*, 2009.

[2] J. Bonwick and B. Moore, "ZFS deduplication," Oracle White Paper, 2010.

[3] M. Gorman, *Understanding the Linux Virtual Memory Manager*, Prentice Hall, 2004.

[4] J. Corbet, "The folio pull request," LWN.net, 2021.

[5] M. Wilcox, "Folios," LWN.net, 2021.

[6] Linux Kernel Documentation, "XArray," Documentation/core-api/xarray.rst.